+ Lecture No. : 1 +

■ **Theory Content to be taught :**

1. Unit 4 : Tree.
2. Tree : Basic Tree Terminologies.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

● Introduction :-

A tree is a non-linear data structure used to represent data containing a hierarchical ~~relationships~~ relationship between elements.

For example : Records, family trees and Table of contents.

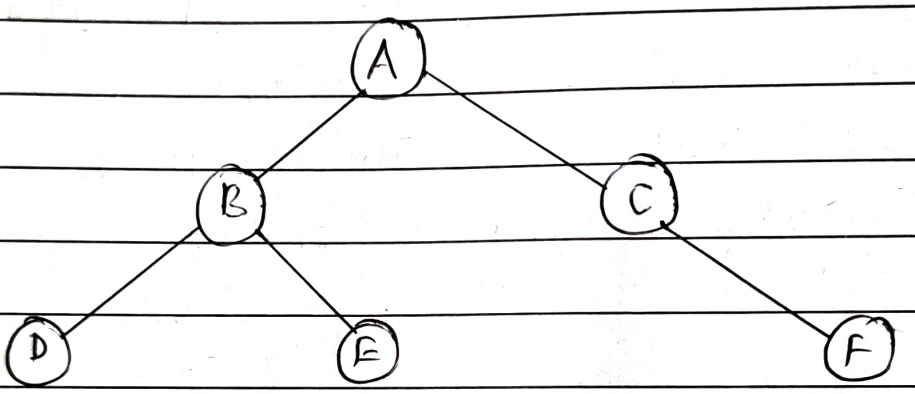
○ Defination :- A tree is a finite set of one or more nodes such that —

i) There is a specially designated node called "root".

ii) The remaining nodes are partitioned into n>=0. disjoint sets T_1 to T_n where each of those sets is a tree.

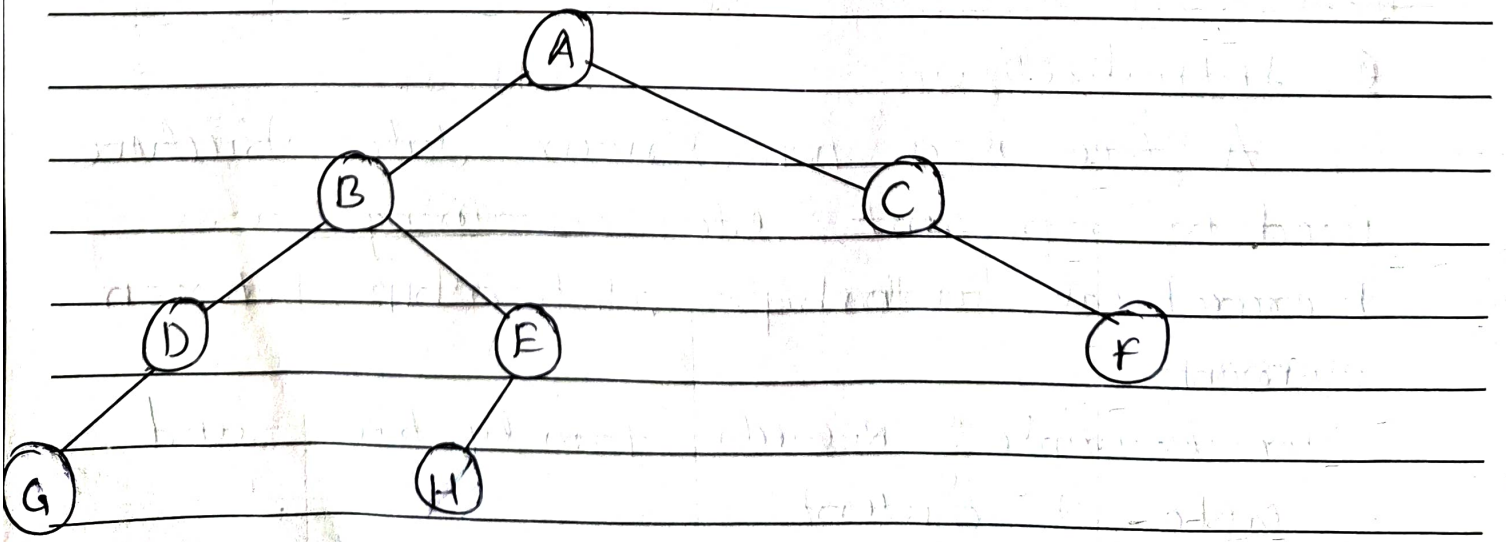
T_1 to T_n are called the sub-trees of the root.

The concept of tree is represented in fig.



⑥ Tree Terminologies :-

Consider the following tree :-



① NULL Tree :-

A tree with no nodes is a NULL Tree.

② Root :-

Root is a unique node in the tree to which further subtrees are attached. This node does not have parent.

In the fig. 'A' is the root node.

③ Node :-

A node of a tree is an item of information along with the branches to other nodes.

In the fig. A, B, C, D, E, F, G & H are all the nodes.

④ parent Node :-

The node having further sub-branches is called parent node.

In fig. 'A' is the parent node of B & C.

⑤ Leaf Node :-

A leaf node is a terminal node of a tree, It does not have any nodes connected to it. All other nodes are called non-leaf nodes or internal nodes.

In the fig. F, G and H are the leaf nodes.

⑥ Degree of a node :-

The total no. of subtrees attached to that

node is called it's ~~subtrees~~ degree.

In the fig. node B has two children. So, degree of B is 2. G, H, and F are nodes of degree 0. A node of degree 0 is also known as leaf node. A leaf node is a terminal node and it has no children.

⑦ Degree of a Tree :-

The degree of a tree is the maximum degree of the nodes in the tree.

In fig. the maximum degree of the shown tree is 2.

⑧ Level of a node :-

The level of a node in the binary tree is defined as :

a) The root of the tree has level 0.

b) The level of any other node in the tree is one more than the level of it's further.

Ex.

Nodes	Level.
A	0
B, C	1
D, E, F	2
G, H	3

(9) Depth or height of a tree :-

The maximum level of any node in the tree is the height or depth of the tree.

In fig. the given tree has a height = 3.

(10) path :-

Path is a sequence of nodes in which each node adjacent to the next node.

In fig. A, B, D, G can be a path.

(11) Ancestor Node :-

The ancestor of a node are all the nodes along the path from the root to that node. In fig. B and A are ancestor of D.

(12) Descendant Node :-

The descendants of a node are all those nodes which are reachable from the node. In fig. D & G are descendants of 'B'.

(13) Siblings :-

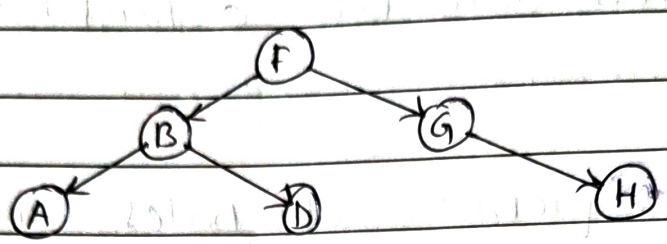
Children of the same parents are called siblings.

In fig. B and C are the siblings. D & E are the siblings.

(14) Directed Edge :-

An edge or link from parent to the child which shows directions is called directed edge.

Ex :-



(15) In-degree :-

In-degree of a node is the number of edges arriving at that node.

The in-degree of a root is, by defination, '0'.

For ex :- In fig. indegree of node B is 1 and indegree of node D is 1. Indegree of root node F is 0.

(16) out-degree :-

Out degree of a node is the number of edges leaving that node.

All nodes in the tree can have zero, one or more branches leaving them, that is they may have out degree of zero, one or more.

For ex :-

node	out degree.
F	2
B	2
G	1
A, D, H	0.

+ Lecture No. : _____ +

■ **Theory Content to be taught :**

1. Different types of Trees : Binary Tree.
2. _____
3. _____
4. _____
5. _____

■ **Specific Objectives :**

1. _____
2. _____
3. _____
4. _____
5. _____

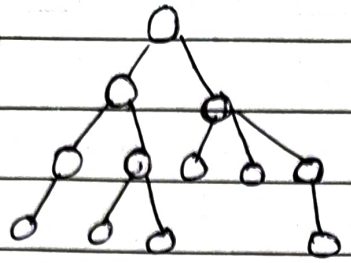
■ **Theory Content :**

© Different Types of Trees :-

① General Tree :-

General trees are unordered tree data structures where the root node has minimum '0' and maximum 'n' subtrees. The general trees have no constraint placed on their hierarchy. The root node thus acts like the superset of all the other subtrees.

Ex :-



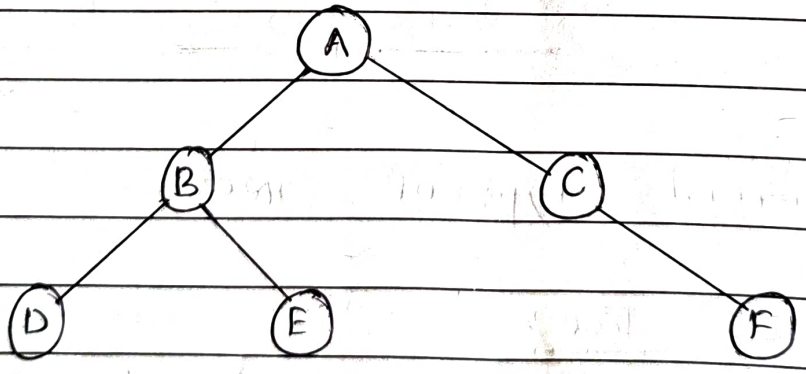
General Tree.

② Binary Tree :-

A binary Tree is a type of tree data structure where each node can have a maximum of two child nodes, a left child node and a right child node.

It is a special type of tree, where node has either no children, one child or two children.

Child of a node in a binary tree on the left is called left child and on the right is called right child.



Operations on Binary Tree :-

a) Creation :- create an empty binary tree to which root points.

② Traversal :- To visit all the nodes in a binary tree.

③ Deletion :- To delete a node from non-empty binary tree.

④ Insertion :- To insert a node into an existing binary tree

⑤ Merge :- To merge two binary tree.

⑥ Copy :- Copy a binary tree.

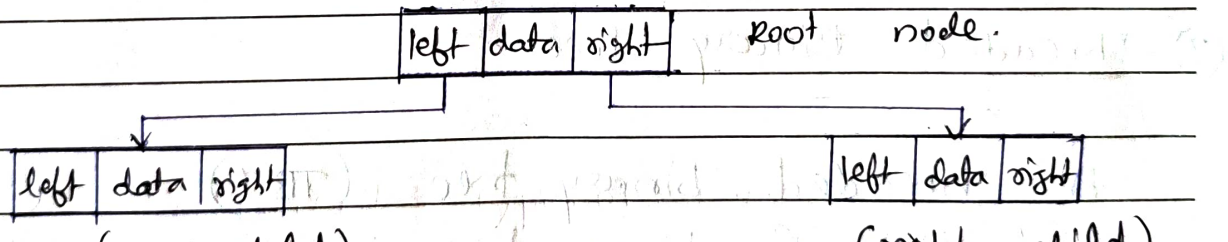
⑦ Compare :- Compare two binary trees.

- Representation of Binary Trees :-

There are two primary ways to represent binary trees.

1. Linked Node Representation :-

This is a simplest way to represent a binary tree. Each node contain data & pointers to it's left and right children.



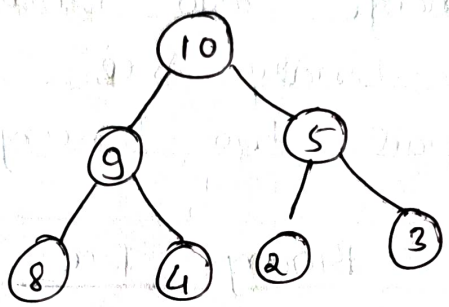
2. Array Representation :-

Array representation is another way to represent binary trees, especially useful when the tree is complete (all levels are fully filled except possibly the last, which is filled from left to right).

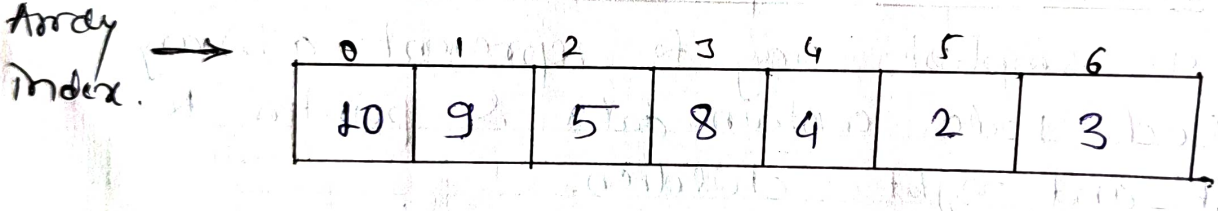
In this method:

- ✓ The tree is stored in an array.
- ✓ For any node at index i:
  - left child: located at $2 * i + 1$.
  - right child: located at $2 * i + 2$.
- ✓ Root node: stored at index 0.

Ex:-



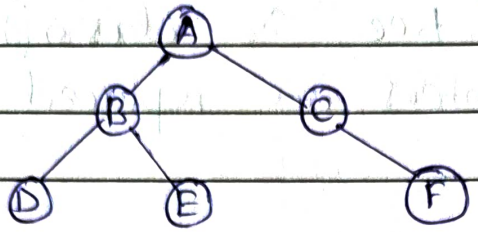
Array Representation:-



* ③ Threaded Binary Tree:-

A threaded binary tree (TBT) is a special type of binary tree in which NULL pointers are replaced by links (threads) to help traverse the tree without using a stack or recursion.

Example of Normal binary Tree.



Inorder Traversal (left Data - right).

D → B → E → A → C → f.

In normal tree, nodes D, E and F have NULL pointers.

Concept of Threading :-

Instead of keeping NULL pointers empty:

- When a node does not have a left child:
  → Its left NULL pointer is used to point to the node that comes just before it in inorder traversal.
  This node is called inorder predecessor.

- When a node does not have a right child:
  → Its right null pointer is used to point to the node that comes just after it in inorder traversal.
  This node is called the inorder successor.

Understanding of Inorder predecessor & successor :-

Understanding with Node E :-

Inorder - D → B → E → A → C → f

For E - Node before E = B → Inorder predecessor.
Node ~~After~~ E = A → ~~Inorder~~ successor.

• Types of threaded binary Tree :-

A threaded binary tree is classified based on which NULL pointers are replaced by threads.

① Left threaded binary tree :-

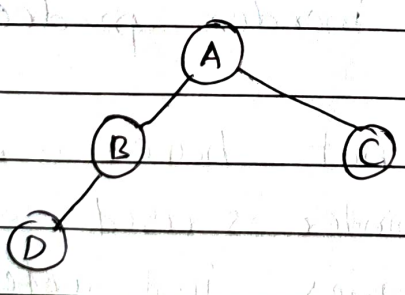
In a left threaded binary tree, only the left NULL pointers are replaced by threads.

- ✓ left thread points to the inorder predecessor.
- ✓ Right pointer behaves normally.

Imp :- left thread is used only when left child is NULL

So:
- ✓ If a node has a left child → No thread used.
- ✓ If a node does not have left child → left pointer becomes thread.

Ex :-



Inorder Traversal = D → B → A → C.

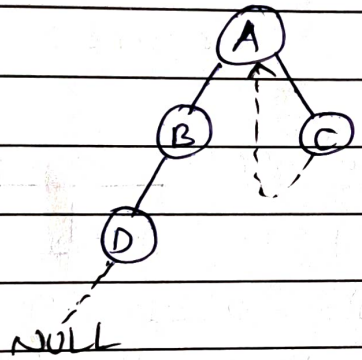
Node 'A' & 'B' have the left child. Node 'D' and 'C' don't have the leftchild node.

In inorder traversal, D is the first node, so it has no predecessor.

So, D → no inorder predecessor → left pointer = NULL (or head).

For node B, has left child D. → ND left thread.
For node A, has left child B → NO left thread.
for node C,
- ✓ No left child.
- ✓ In inorder sequence, predecessor of C is A.

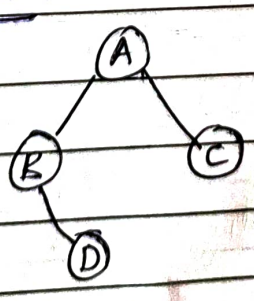
So, C.left ⇒ A
C.righ ⇒ NULL.



② Right threaded Binary Tree :-

In a right threaded Binary tree, only the right null pointers are replaced by threads.
- ✓ right thread → points to inorder successor.
- ✓ left pointer → normal child pointer.

Ex:- Given tree :-



Inorder Traversal ⇒ B → D → A → C.

In a right threaded binary tree, if a node has no right child, it's right pointer points to it's inorder successor.

→ for Node 'B'.

✓ No left child → not needed for right thread.

✓ Has a right child D → no right thread.

For Node 'D' →

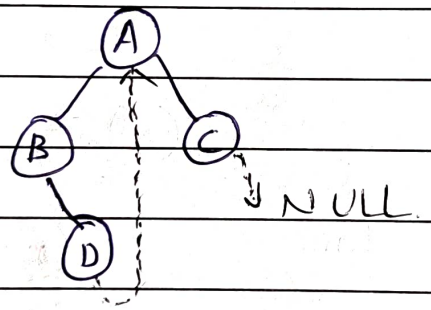
- ✓ No right child.
- ✓ In inorder sequence, successor of D is A.

for Node 'A' →

- ✓ Has a right child C → No thread.

for node 'C' →

- ✓ No right child.
- ✓ In inorder sequence, no successor for 'C'. So it is considered as NULL.

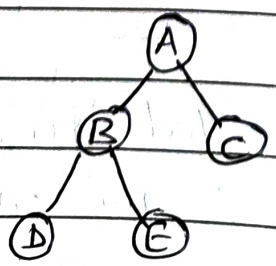


③ Double threaded Binary Tree :-

In a double threaded binary tree, both left and right NULL pointers are replaced by thread.

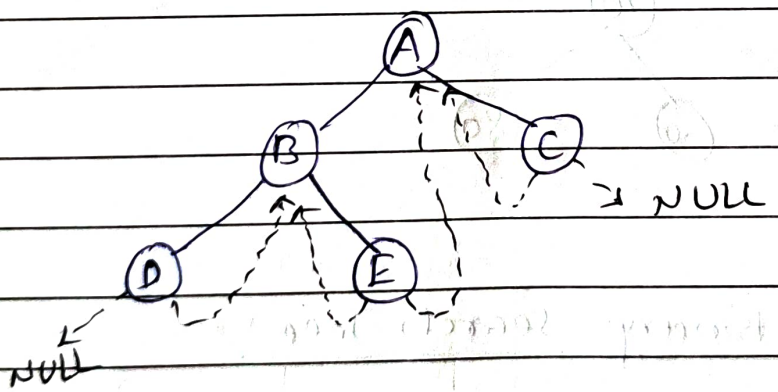
- ✓ left thread → inorder predecessor
- ✓ right thread → Inorder successor.

Ex :-



Inorder Traversal :- D → B → E → A → C.

- For Node 'D' →
  - ✓ No left child → predecessor = NULL.
  - ✓ No right child → successor = B.
- For Node 'B' →
  - ✓ Has left child D → No left thread.
  - ✓ Has right child E → No right thread.
- For Node 'E' →
  - ✓ No left child → predecessor = B.
  - ✓ No right child → successor = A.
- For Node 'A' →
  - ✓ Has left child B → No left thread.
  - ✓ Has right child C → No right thread.



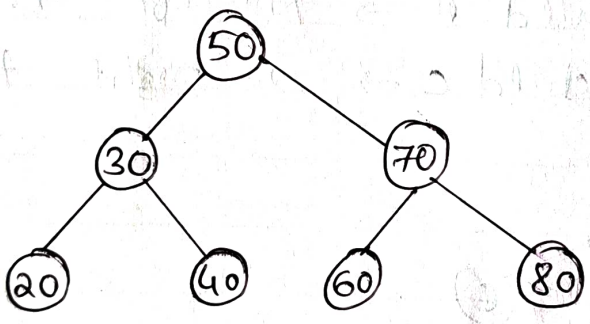
- For Node 'C' →
  - ✓ No left child → predecessor = A
  - ✓ No right child → NULL.

③ Binary search Tree :-

A binary search tree (BST) is a binary tree in which the nodes are arranged according to their values.

The left node has a value less than its parent and the right node has a value greater than or equal to the parent node. i.e. all nodes in the left subtree have values less than the root and those in the right subtree have values greater than or equal to the root.

Ex :-



- properties of Binary search Tree :-

1. left subtree contains smaller value.
2. Right subtree contains greater value.
3. Each subtree is also a BST.
4. Inorder traversal of BST gives sorted order.

• operations on Binary search Tree :-

(A) search operation :-

Searching a node in BST.

steps :-

1. start from root
2. if value = root → found.
3. if value < root → go left.
4. if value > root → go right.

(B) Insertion operation :-

To insert a node in the binary search tree.

steps :-

1. Compare with root
2. Move left if smaller
3. Move right if greater
4. Insert at NULL position.

(C) Delete operation :-

To delete a node from the BST.

There are 3 cases -

1. leaf Node (No child) - Just delete node.
2. One child - replace node with it's child.
2. Two children -

Replace with -

✓ Inorder successor. (smallest in right subtree)

or ✓ Inorder predecessor. (largest in left subtree).

- Advantages of BST :-
  1) Fast searching
  2) Easy insertion & deletion.
  3) Sorted output using inorder traversal.
  4) Dynamic structure (grows / shrinks easily).

- Disadvantages of BST :-
  1) A BST can become skewed if elements are inserted in sorted order
  2) Extra memory for pointers. - Each node requires left pointer & right pointer. Extra memory overhead compared to arrays.
  3) Complex deletion operation.

+ Lecture No. :

■ **Theory Content to be taught :**

1. Applications of Binary Trees.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

- Applications of Binary Trees :-

Binary trees are widely used in computer science because they provide an efficient way to store, organize and access data.

1) Expression Tree :- (Arithmetic Expressions)

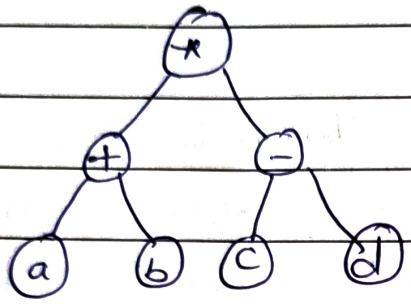
A binary tree used to represent mathematical expressions.

✓ Internal nodes → operators (+, −, ×, /).

✓ leaf nodes → operands (numbers / variables).

Ex:- Expression →

$(a+b) * (c-d)$

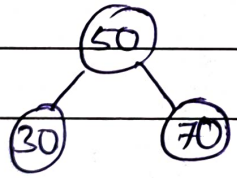


Applications:- ✓ Compiler design.
✓ evaluation of expressions

2) Binary Search Tree (BST):-

Binary tree used for fast searching, insertion, deletion.

Ex:-



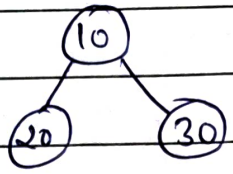
Applications:- ✓ Dictionary implementation.
✓ Database Indexing.
✓ searching large dataset.

3) Heap (priority Queue):-

A binary tree used to implement priority Queue.

Types: * max heap → largest element at root
* min heap → smallest element at root.

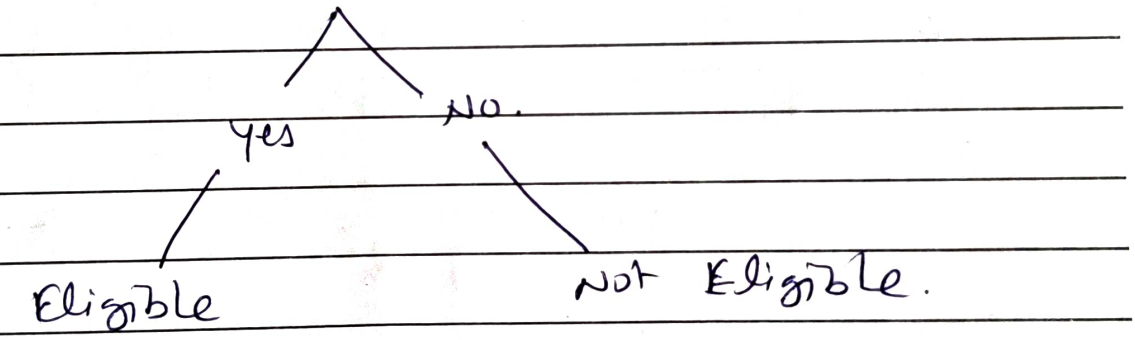
Ex:- min heap.



Applications :- ✓ CPU scheduling.
✓ Heap sort

4) Decision Trees :- (AI / machine learning).
A binary tree used for decision making process.

Ex :-



Applications :-
✓ M/c learning classifications.
✓ Business decision models.

Binary Tree Traversal :-

Tree traversal refers to the process of visiting or accessing each node of a tree exactly once in a specific order.

Tree Traversal Techniques :-

1) In-order Traversal :-

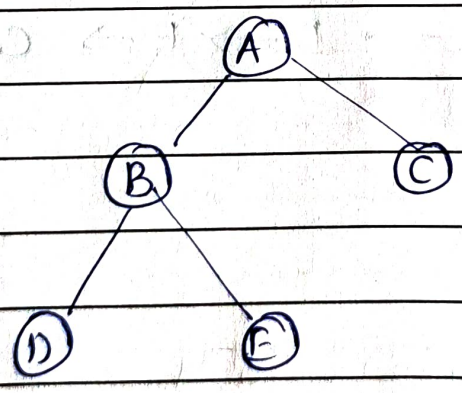
In order traversal visits the node in the order - Left → Root → Right.

Algorithm :-

Inorder (tree)

1. Traverse the left subtree, i.e. call Inorder (left → subtree)
2. Visit the root
3. Traverse the right subtree, i.e. call inorder (~~left~~ right → subtree)

Ex :-



In-order ⇒ D → B → E → A → C .

2] Preorder Traversal :-

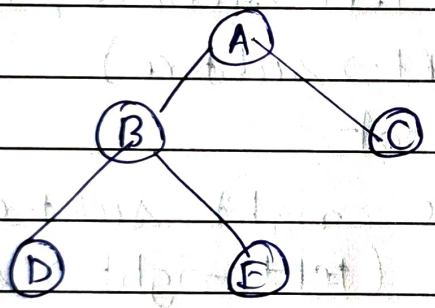
preorder traversal visits the node in the order : Root → left → right.

Algorithm :-

preorder (tree) :

1. visit the root
2. Traverse the left subtree, i.e. call preorder (left → subtree)
3. Traverse the right subtree, i.e. call preorder (right → subtree).

Ex :-



preorder :- A → B → D → E → C.

3]. Postorder Traversal :-

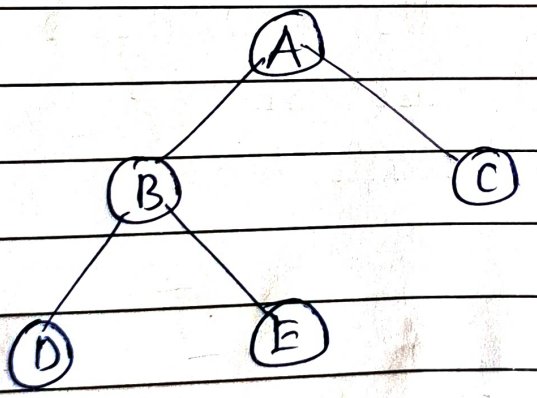
postorder Traversal visits the node in the order : Left → Right → Visit.

Algorithm :-

postorder (tree) :

1. Traverse the left subtree, i.e. call postorder (left → subtree).
2. Traverse the right subtree, i.e. call postorder (~~left →~~ right → subtree)
3. Visit the root.

Ex :-



postorder ⇒ D → E → B → C → A.

✦ Lecture No. : _____ ✦

■ **Theory Content to be taught :**

1. Tree Variants : AVL Tree
2. Height of an AVL tree.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

● AVL Tree :-

An AVL Tree is a self-balancing binary search tree (BST). It was introduced by Addson-Velsky and Landis in 1962, hence the name AVL.

In an AVL tree, the difference between the heights of the left and right subtrees of any node cannot be more than 1.

This property ensures that the

tree remains balanced, making searching, insertion, and deletion operations efficient.

- AVL Tree definition:-

An AVL tree is a binary search tree in which -

Height (left-subtree) - Height (rgh subtree) ≤ 1 for every node.

This difference is also called balance factor.

- Height of a node :-

Height = No. of edges in the longest path from the node to a leaf.

- How to calculate left height & right height in an AVL tree :

The easiest way is to calculate left / right height from the bottom (leaf nodes) to the top (root).

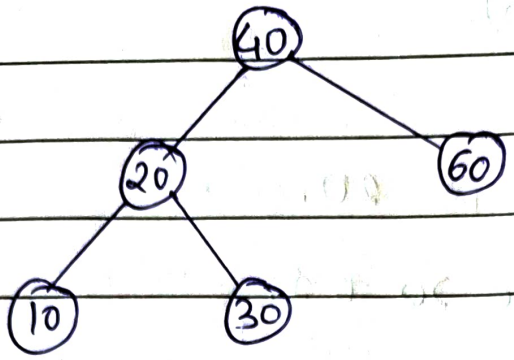
Height Rules :-

- leaf node height = 0
- Empty (NULL) child height = -1.

for any node -

Height (node) = 1 + max(Height (left), Height (right))

- Calculate Heights - Example:



Step 1: Calculate Heights of Leaf nodes.

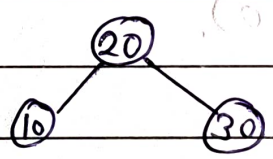
Leaf nodes are - 10, 30, and 60.

10 = 0

30 = 0

60 = 0. because they have no children.

Step 2: Calculate Height of Node 20.



The left child is 10.

Height of node 10 = 0. So, left Height = 0.

The right child is 30

Height of node 30 = 0. So, right height = 0.

Now calculate the height of node 20.

Height (20) = 1 + max (left height, right height)

= 1 + max (0,0)

= 1.

So, Node 20 height = 1.

Step 3: Calculate Height of Node 40.

Left Height of 40 :

The left child is 20.

Height of node 20 = 1.

So, left Height = 1.

Right height of 40 :

The right child is 60.

Height of node 60 = 0.

So, right height = 0.

Now, calculate the height of node 40.

Height (40) = 1 + max(1, 0).

= 1 + 1

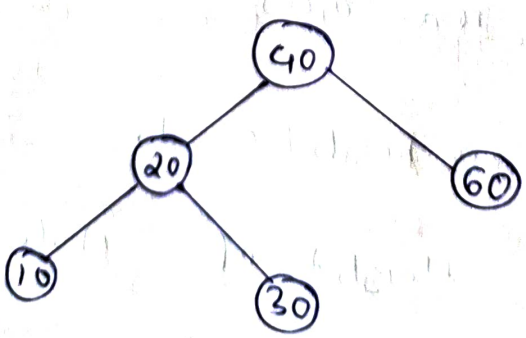
= 2.

- How to calculate the Balance factor :-

Once you know the left and right heights, use :

Balance factor = Left height − Right height.

Example :-



Calculate balance factor of each node.

Node 10, No left & right child.

So, left height = -1 and right height = -1.

BF = left height - right height

= (-1) - (-1).

= 0.

Node 10 is balanced.

Node 30, No left & right child.

So, BF = 0.

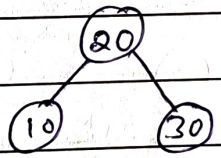
Node 30 is balanced.

Node 60, No left & right child.

So, BF = 0.

Node 60 is balanced.

Node 20,



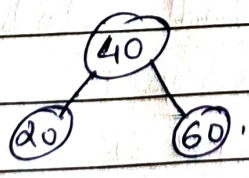
Height of left child (10) = 0

Height of right child (30) = 0.

So, BF = 0 - 0 = 0.

Node 20 is balanced.

Node 40,



Height of left child (20) = 1.

Height of right child (60) = 0.

BF = left height - right height.

= 1 - 0.

= +1.

Node 40 is also balanced, because Balance factor of +1 is allowed in an AVL tree.

- Important :-
  - ✓ BF = 0 → perfectly balanced.
  - ✓ BF = +1 → left subtree is one level taller. (still balanced).
  - ✓ BF = -1 → right subtree is one level taller. (still balanced).
  - ✓ BF = +2 or -2 → Tree is unbalanced and requires a rotation.

+ Lecture No. : +

■ **Theory Content to be taught :**

1. AVL tree Rotations.
2. - Single & Double Rotations.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

① AVL tree Rotations :-

The AVL Tree must always remain balanced. The balance factor (BF) of every node must be

BF = left Height - Right Height.

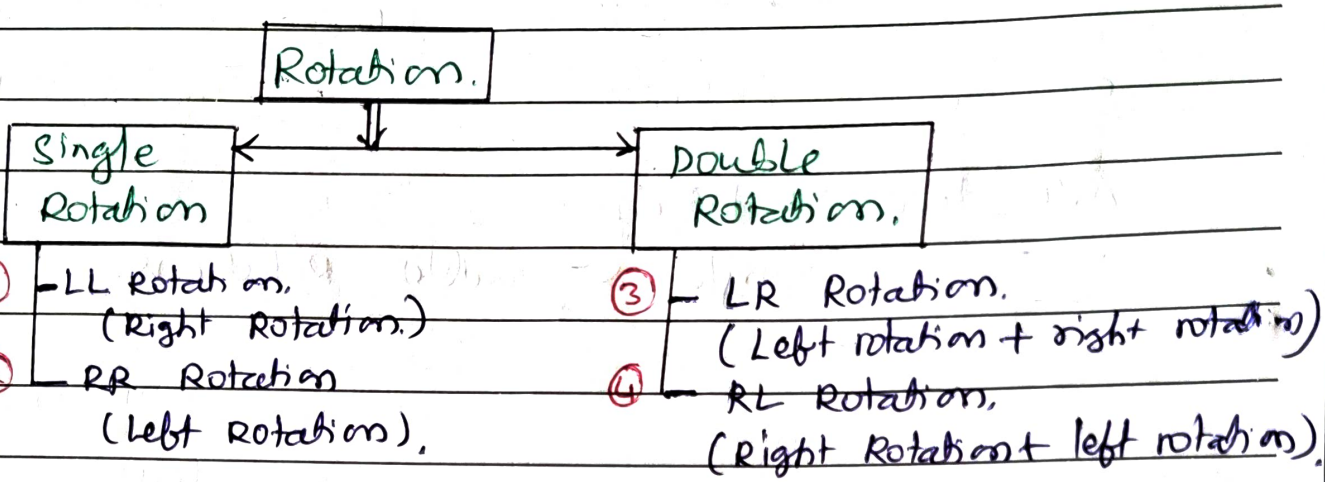
Allowed values are - -1, 0, +1.

If the BF becomes, +2 or -2 the tree becomes unbalanced.

To make it balanced again, we perform "Rotation" A rotation changes the structure of the tree

Without changing the binary search tree (BST) property.

- Types of Rotation :-



- Single Rotation :-

A single Rotation means only one rotation is performed.

There are two types -

① LL Rotation (Left-Left Rotation) :-

When does LL Rotation Occur?

LL rotation occurs when -

- ✓ The tree becomes left heavy.
- ✓ A new node is inserted into the left subtree of the left child.

This creates : BF = +2.

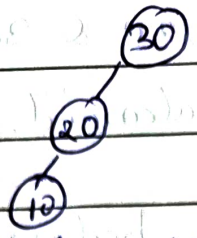
Example: Insert the following node in BST. 30, 20 & 10.

first (30) → it is balanced.

Now 20 → (30) → it is balanced.

(20)

Now 10 →



Now calculate the balance factor (BF).

Node 10, leaf node, here Height = 0.

So, BF = 0.

Node 20, left height = 0., Right Height = -1.

BF = 0 - (-1) = +1. (Balanced).

Node 30, left height = 1, Right height = -1.

BF = 1 - (-1) = +2.

Node 30 becomes unbalanced.

⇒ Why it is called LL?

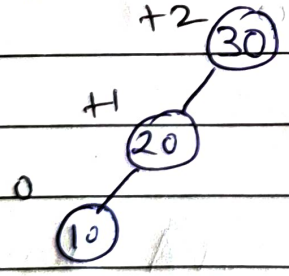
The new node is inserted.

✓ left of 30 ✓ left of 20.

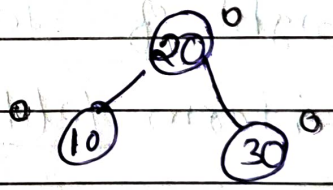
Hence, LL Rotation.

⇒ Here perform Right rotation.

Before Rotation -



After Rotation -



② RR Rotation (Right - Right Rotation).

When does RR Rotation occure?

RR rotation occurs when -

✓ Tree becomes right heavy

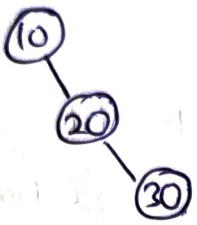
✓ Node is inserted into the right subtree of the right child.

BF becomes = -2.

Example – Insert nodes, 10, 20 & 30.

first 10 ⇒ (10) ⇒ It is balanced.

insert 20 ⇒ ⇒ It is balanced.

Insert 30 ⇒  ⇒

Heights ⇒ 30 = 0
20 = 1
10 = 2.

Balance factor –

Node ⇒ (20). Here LH = -1, RH = 0.

BF = -1 (Balanced).

Node ⇒ (10). Here LH = -1, RH = 1.

BF = (-1) - (1) = -2. (Unbalanced)

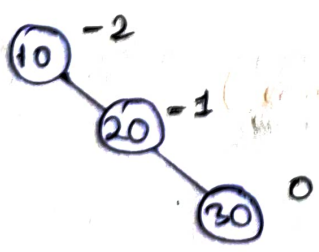
⇒ Why it is called RR ?

The new node is inserted.

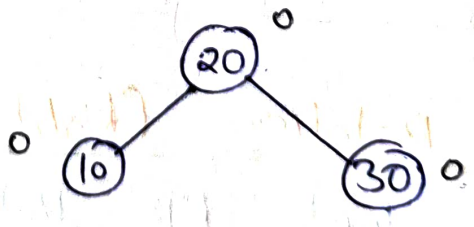
✓ right of 10 ✓ right of 20.

⇒ Hence perform <u>left Rotation</u> :–

Before Rotation –



After Rotation :–



Double Rotation :-

Sometimes a single rotation is not enough. We need two rotations. These are called "Double Rotation".
There are two types :-

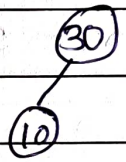
① LR Rotation (Left - Right Rotation) :-

When does LR Rotation Occur ?
Tree becomes left heavy BUT Insertion is in the right subtree of the left child.

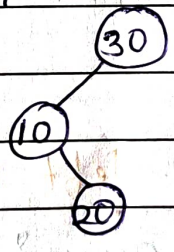
Example - Insert node 30, 10, 20.

first, insert 20 ⇒

next, insert 10 ⇒



next, insert 20 ⇒

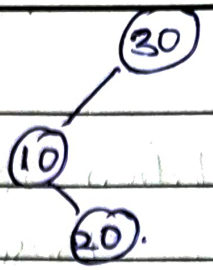


Balance factor ⇒
node 10 ⇒ LH = -1, RH = 0.
BF = -1. (Balanced)
node 30 ⇒ LH = 1, RH = -1.
BF = 1 - (-1) = 2 (Unbalanced).

⇒ Why it is called LR ?
The new node is inserted -
✓ left of 30 ✓ right of 10.

⇒ Hence perform Left - right Rotation.

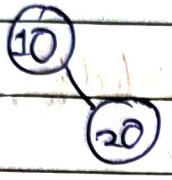
Before Rotation -



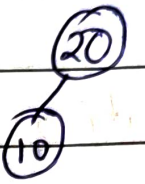
step 1:

left rotation of node 10.

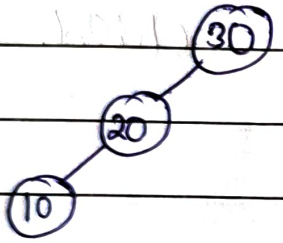
Before -



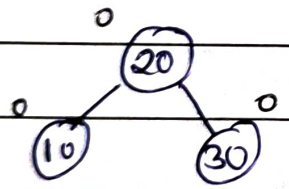
After -



Now whole tree -



step 2: Right rotation on 30.



⇒ balanced.

(2) RL Rotation (Right-left Rotation) :-

When does RL Rotation occur?

✓ Tree becomes right heavy BUT Insertion happens in the left subtree of right child.

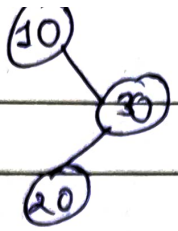
Example - Insert node 10, 30 and 20

first insert 10,

Now insert 30,

⇒ Balanced.

Now insert 20.



Balance factor ⇒

Node 30, LH = 0, RH = -1.

BF = 1. ⇒ Balanced.

Node 10, LH = -1, RH = 1.

BF = (-1) - 1 = -2. ⇒ Unbalanced.

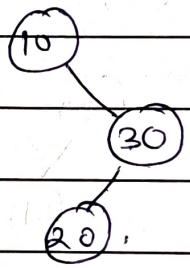
⇒ Why it is called RL ?

The new node is inserted –

✓ Right of 10 ✓ left of 30.

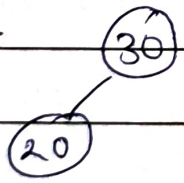
⇒ Hence perform RL Rotation.

Before Rotation –

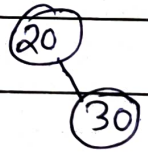


step 1: Right rotation on 30.

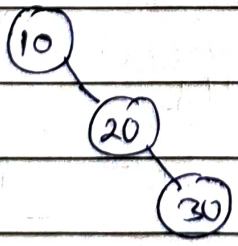
Before –



After –

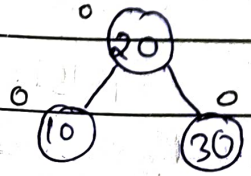


Now whole tree –



step 2 :-

Left rotation on 10.



⇒ balanced.

■ **Theory Content to be taught :**

1. AVL Tree : Insertion, deletion of node
2. in AVL Trees.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

AVL Tree : Insertion, deletion of node in AVL Tree :-

Insertion in an AVL tree consists of four main steps -

step 1 : Insert the new node.

Insert the new node exactly as in a Binary search Tree.

step 2 : Update Heights.

After insertion, move back toward the root

and update the height of every ancestor.

step 3: Find balance factor.

For every ancestor,

BF = Height (left) − Height (right).

If BF becomes − +2 or −2, the tree is unbalanced.

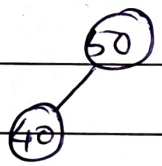
step 4: Perform Rotation.

Depending upon where the insertion occurred, perform one of four rotations.

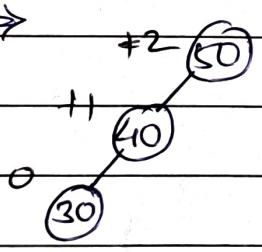
Example :- Insert nodes, 50 - 40 - 30 - 20 - 10.

Insert 50.

⇒

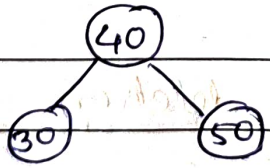
Insert 40 ⇒ 

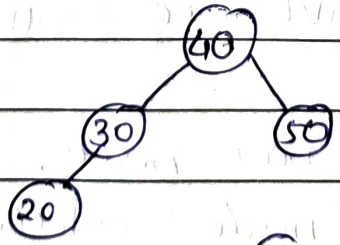
⇒ Balanced.

Insert 30 ⇒ 

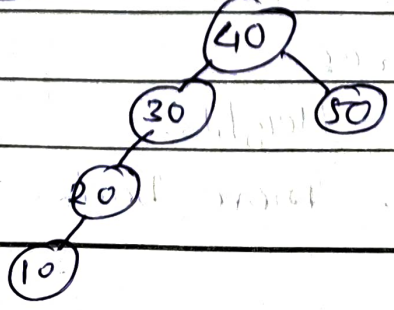
⇒ Unbalanced.
This is the LL case.

After LL Rotation. —



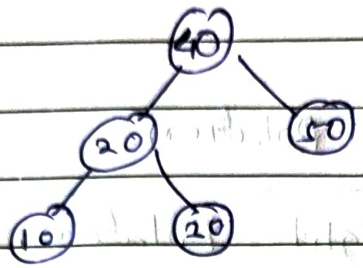
Insert 20 ⇒ 

⇒ Balanced.

Insert 10 ⇒ 

node 20 becomes
⇒ Unbalanced.
This is LL case

Right rotation on 30 →



- **AVL Tree Deletion :-**

Deletion is similar to BST.

After deletion —

- ✓ Update heights.
- ✓ Calculate Balance factor.
- ✓ perform rotations if required.

Deletion may require multiple rotations.

- **BST Deletion Rules :-**

Before balancing, delete the nodes as in a normal BST.

Steps in AVL tree Deletion :-

Step 1 : BST deletion.

Delete the node using normal BST rules :

- ✓ leaf node → remove directly.
- ✓ One child → replace with child.
- ✓ Two children → replace with inorder successor (or predecessor).

Step 2 : Update heights.

After deletion, update height from bottom to top.

step 3 : check Balance factor (BF).

If BF = -1, 0, +1 ⇒ Ok.

BF = -2 or +2 ⇒ Unbalanced.

step 4 : perform Rotation. (if needed).

case	Rotation.
LL	Right Rotation.
RR.	Left Rotation.
LR	Left + Right Rotation.
RL	Right + Left Rotation.